

## 商家后台与可观测性 (v2 扩展版)

从五角色视角看一个商家入驻 / 一次客诉定位 / 一笔退款 / 一次大促 SLO 守护

---

gomall · 业务侧 deck

业务视角先行：商家后台 + 可观测对谁有什么用

商家数据现状与 merchant 角色路线图

merchant API 草案与入驻流程

商家看板：4 象限信息架构

Jaeger：链路追踪解决的真实问题

Prometheus 业务 metrics + Grafana 大盘

ELK / EFK：日志集中化的工程化路线

静默降级 + SLO + trace 驱动决策

业务视角先行：商家后台 + 可观测对  
谁有什么用

---

# 五个角色，五个截然不同的问题

## C 端用户

”我的订单  
怎么还没发”  
不关心系  
统，只看结果

## 商家

”我今天  
卖了多少”  
”哪个 SKU  
要补货”

## 运营 / 平台

”GMV 实时多少”  
”商家健康度  
/ 平台健康度”

## 客服

”用户投  
诉这一单”  
”我能查  
到全链路”

## SRE

”99.95% SLO 还  
剩多少预算”  
”P0 告警  
5min 上线”

同一套底层信号

(trace + metrics + log + 业务表 BossID) 给五个角色各自的”工具箱”。本 deck 不讲”Prometheus 怎么部署”，讲每个角色拿到这套工具能解决自己的什么业务问题。

*repository/db/model/order.go:12 BossID* 数据基础已就位；五张表 *BossID* 见下一帧

## 商家入驻的业务价值：先做基础再做看板的取舍

- **业务驱动**：每多一个商家入驻，平台多一份 GMV 抽佣（典型 5%–10%）。”商家招得多 + 商家留得住”是 GMV 第一性增长。
- **招商团队的诉求**：能给商家承诺三件事——(a) 我能看到自己卖了多少；(b) 我能看到哪些货要补；(c) 我能自助提现。这三件事直接决定**签约转化率**。
- **BossID 现在做了什么**：5 张核心表（order / product / cart / favorite / skill\_product）的 BossID 字段都已经在了——**数据基础完整**，单店数据隔离的物理前提满足。
- **BossID 现在还差什么**：/api/v1/merchant/\* 一个路由都没有；没有 merchant role；没有商家自助 API。**有数据没出口**。
- **为什么先做 BossID 再做看板**：BossID 是数据合约（schema），改起来牵连大；看板是查询出口（handler），改起来轻。先把硬骨头啃了，看板路线图就只是补 handler。

## 商家三类自助看板：路线图与业务取舍

### 销售看板

GMV / 退货率  
SKU 排名 /  
转化漏斗  
P1 优先级·  
招商必备

### 库存看板

缺货告警 / 周转率  
滞销 SKU /  
补货建议  
P2 优先级·  
留存关键

### 财务看板

到账金额 /  
提现可用  
对账明细 /  
月度结算单  
P0 优先级·  
资金信任

- 为什么财务 **P0**：商家最敏感的是钱什么时候到。”看不见钱 = 不信平台”，留存崩盘。
- 为什么销售 **P1**：决定签约时的”产品 pitch 能不能讲赢”，是招商团队的弹药。
- 为什么库存 **P2**：缺货只是少卖，不会让商家提桶跑路；可以靠”事件 + 邮件”先兜，看板后补。
- 分期路线图：阶段 1 财务 API + 月度对账 PDF；阶段 2 销售实时聚合（Redis 60s 缓存）；阶段 3 库存告警 ZSET + 周转率离线计算。每阶段单独可上线。

## 可观测分层：每一层解决一个角色的一个问题

### 链路追踪 Jaeger

”这一笔请求  
慢在哪一段”  
→ 客服 + SRE  
个体定位

### Metrics

#### Prometheus

”系统整体健康度”  
→ SRE on-  
call 总览

### 日志 ELK

”客服查这单  
用户做了什么”  
→ 客服投诉处理

- 分层不是堆工具，是按”问题”分工。一种工具回答一类问题，混用会让 SRE 在 metrics 大盘里找具体订单（错位）。
- **trace** 回答个体问题：用户报订单 #12345 卡了 5 秒 → 客服贴 traceId 给 SRE → Jaeger 看 span 树 → 一眼看到 db.Insert 38ms（见后续 trace span 树帧）。
- **metrics** 回答总体问题：下单 p99 是否过阈值？错误率是否飙升？→ 用 Counter / Histogram / Gauge 在大盘呈现。
- **log** 回答取证问题：客服需要”按用户号查这天他做了什么 / 报错码是什么 / 上下游调用链是什么” → ELK 全文检索 + traceId 关联。

## 静默降级的业务价值：商家慢 5 分钟 vs 用户搜不到

- 业务的不对称：电商两端”出错容忍度”截然不同。
  - C 端用户搜不到结果 → 立刻跳竞品。不可接受。
  - 商家新上 SKU 到搜索可见慢 5 分钟 → 客户能接受，BD 可以解释。
- 所以系统取舍：保 C 端可读的优先级 > 保商家写最新可见的优先级。这是 README 技术亮点 #11 ”外部依赖不可用主流程不挂”的业务解释。
- 落地：cmd/main.go 4 个 tryInitX (RabbitMQ / ES / Web3 / Milvus) 任一挂 → HTTP 正常起；ES 挂 → 搜索退 DB LIKE 仍可用；Milvus 挂 → 语义召回关，关键词搜仍可用。
- 对各角色的解释：
  - C 端用户：完全无感（看到的是结果质量轻微下降）
  - 商家：BD 接到”新 SKU 搜不到”投诉 → 5-10min 后自然恢复（无需介入）
  - SRE：Warn 日志聚集触发 P2/P3 告警 → 工作时间内排查
  - 客服：拿到 traceId → ELK 一搜看到 ”ES 不可用，搜索降级到 DB” Warn → 当场答客户



## 商家数据现状与 merchant 角色路线 图

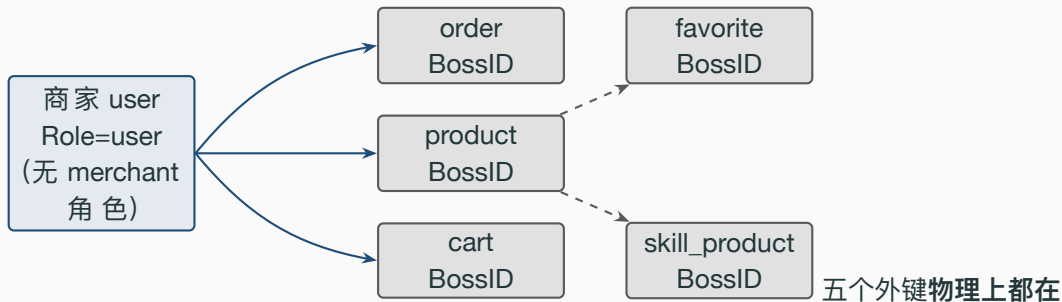
---

## 一个商家在 gomall 里能干什么、不能干什么

- **业务背景：**招商团队签约时给商家承诺三件事——看营收 / 看库存 / 看到账。现状只有”账号”和”上架”两件事能做，承诺兑现率 0/3 → 签约转化率拉低。
- **能干：**注册账号、上架商品、被买家下单、被买家收藏。
- **不能干：**查”我的今日营收”、查”我的待发货”、查”我的库存告警”、自助退款、自助提现。
- 五张核心表里 **BossID** 字段早就埋好了：order.BossID、product.BossID、cart.BossID、favorite.BossID、skill\_product.BossID。
- 但是 /api/v1/merchant/\* 一个路由都没有——数据有、查询入口缺。
- **业务解读：**把”数据基础完整 + API 缺位”的现状**诚实摆出来**，对招聘官 / 答辩评委来说更值钱——你知道现在差什么、知道按什么顺序补，比”什么都做了”的清单更可信。

repository/db/model/order.go:12、product.go:23、cart.go:10、favorite.go:12 五处 BossID；README 业务边界”  
没有 merchant role”

## BossID 散落在五张表里

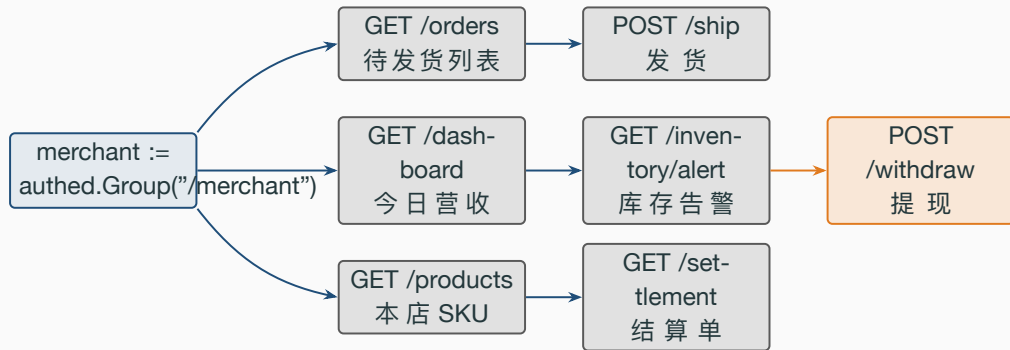


了，只是逻辑上没有 **merchant API** 把它们串起来。补一个 **merchant group** 不需要改 schema，只是补 5-10 个 handler。*repository/db/model/order.go:12*、*product.go:23*、*cart.go:10*、*favorite.go:12*

## merchant API 草案与入驻流程

---

# merchant Group 总览



7 个接口分三层：查 (**dashboard / orders / products / inventory / settlement**)、操作 (**ship**)、资金 (**withdraw**)。资金类需要二次鉴权 (短信/邮箱 OTP)。 *routes/router.go:157-164*  
*admin* 子组模板, *merchant* 完全复用该姿势

## 7 个接口的签名与权限矩阵

| 路径                        | 方法   | 入参                         | 返回          | 备注                  |
|---------------------------|------|----------------------------|-------------|---------------------|
| /merchant/dashboard       | GET  | date=YYYY-MM-DD            | DashboardVO | 命 Redis 60s 缓存      |
| /merchant/orders          | GET  | status=wait_ship,page,size | []OrderVO   | 默认 status=wait_ship |
| /merchant/orders/ship     | POST | order_id, ship_no, carrier | status      | 幂等 + 写 outbox       |
| /merchant/products        | GET  | on_sale=true, page, size   | []ProductVO | WHERE boss_id=?     |
| /merchant/inventory/alert | GET  | threshold=10               | []SkuAlert  | ZRANGEBYSCORE       |
| /merchant/settlement      | GET  | month=YYYY-MM              | SettleVO    | 上月汇总, T+30          |
| /merchant/withdraw        | POST | amount, bank_acct          | OrderNum    | OTP + 风控 + 异步       |

**权限统一：**RequireRole("merchant","admin") ——admin 可在出问题时代为操作，便于客服兜底。**数据隔离：**所有查询 SQL 末尾强制 AND boss\_id=ctx.UserId，由 service 层注入，handler 不接受 boss\_id 入参。 repository/db/dao/product.go:67-72 已有 WHERE id=? AND boss\_id=? 模板可复用

## merchant.Group 注册：18 行代码

Listing 1: *routes/router.go* *merchant.Group* 草案；与 *admin.Group* 同构

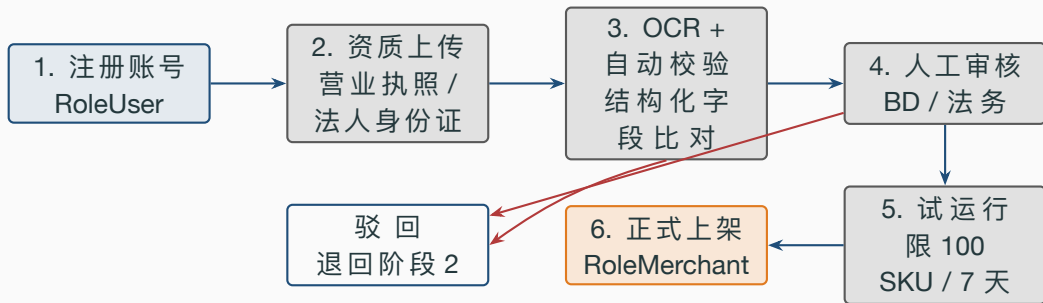
```
157 // 商家后台（新增）—— 与 admin 子组同级，
158 // 资金类接口再叠一层 OTP，由 service 层校验
159 merchant := authed.Group("/merchant")
160 merchant.Use(middleware.RequireRole("merchant", "admin"))
161 {
162     merchant.GET("dashboard", api.MerchantDashboardHandler())
163     merchant.GET("orders", api.MerchantListOrdersHandler())
164     merchant.POST("orders/ship",
165         middleware.Idempotency(),
166         api.MerchantShipOrderHandler())
167     merchant.GET("products", api.MerchantListProductsHandler())
168     merchant.GET("inventory/alert",
169         api.MerchantInventoryAlertHandler())
170     merchant.GET("settlement",
171         api.MerchantSettlementHandler())
172     merchant.POST("withdraw",
173         middleware.Idempotency(),
174         api.MerchantWithdrawHandler())
175 }
```

- **第 3 行 Group("/merchant")**: 与 admin 子组同级挂在 authed 下, 自动继承 AuthMiddleware() ——没登录的请求根本进不到 RequireRole。
- **第 4 行 RequireRole("merchant","admin")**: 变长参数复用, admin 自动有 merchant 权限, 避免”测试一个商家场景要新建一个用户”的反复切换。
- **第 9 行 ship / 第 17 行 withdraw 加 Idempotency()**: 两类产生外部副作用的接口 (生成运单号、扣账户余额); 查询接口不需要幂等。
- **未加 SlidingWindow**: 商家后台是低频管理界面, 单商家 QPS 几乎不会触发限流; 加了反而增加 Redis 压力。
- **未在路由层做 boss\_id 校验**: boss\_id 来自 ctl.GetUserInfo(ctx).Id, 绝不从请求 body / query 读, 避免越权。

*middleware/auth.go AuthMiddleware; middleware/rbac.go:48-86 RequireRole; middleware/idempotency.go*



## 从注册到正式上架：5 个阶段



整个流程平均 **3-5 个工作日**：OCR 即时返回，人工审核占大头（24-48h）；试运行用于发现资质合规但实际运营粗糙的卖家。建议新增表 `merchant_apply(user_id, license_no, status, reviewer_id, ...)`；`service/merchant_apply.go` 待建

## 阶段 2-3：资质上传与 OCR 字段校验

- **上传走 OSS**：POST /merchant/apply/upload 复用 api/v1/upload.go 的 OSS 上传链路，返回 URL。
- **OCR 拉的字段**：营业执照号、企业名称、法人姓名、注册资本、经营范围。
- **自动校验三件套**：(a) 营业执照号 18 位且校验位合法；(b) 法人姓名与身份证 OCR 一致；(c) 申请人手机号实名认证与法人姓名一致（运营商接口）。
- **三项任一不过** → **status=auto\_rejected**，直接返回错误码 80001 " 您的店铺还在审核"，但内部状态记 auto\_rejected\_reason=" 法人姓名不一致"，客服可看不能改。
- **OCR 服务挂了怎么办**：降级为" 手动录入字段 + 人工目检图片"，不阻塞流程；与 deck 10 静默降级哲学一脉相承。

api/v1/upload.go OSS 上传链路；建议新建 service/ocr.go 抽象 OCR 供应商

## 阶段 4-5：人工审核与试运行

- **审核界面**：admin 后台新增 `/admin/merchant/apply/list` + `/admin/merchant/apply/decide`。BD / 法务双签：BD 看品类匹配度，法务看资质合规。
- **审核 SLA**：48 小时内一审。超时进入“催办池”，自动 主管。
- **试运行限额**：通过审核后状态进入 `trial`，`RoleMerchant` 已生效，但隐性加上 (a) SKU 数  $\leq 100$ ，(b) 单日订单  $\leq 500$ ，(c) 不能领大额营销资源位。
- **试运行毕业条件**：7 天内 (a) 无客诉升级 (b)  $DSR \geq 4.5$  (c) 履约率  $\geq 95\%$ 。三项达标自动升 `normal` 状态。
- **异常退出**：试运行期 P0/P1 客诉  $\rightarrow$  状态回退 `trial_failed`，BD 介入复盘。

建议复用 `service/admin.go:46-62 PromoteToAdmin` 模板做 `PromoteToMerchant`；试运行限额可在 `RBAC` 之后挂一个 `trial` 中间件

## 商家看板：4 象限信息架构

---

## 商家看板首屏的 4 个象限

### 今日营收

GMV / 单数 / 客单价  
order WHERE  
Type=2

### 待发货

超 24h 红字  
order WHERE  
status=wait\_ship

### 库存告警

< 商家自设阈值  
product.Num

### 客诉工单

未结案 + 今日新增  
ticket (待建)

4 个象限对应商家一天打开后台的

**4 个动作：**(a) 看赚了多少钱（左上），(b) 看要发什么（右上），(c) 看要补什么货（左下），(d) 看要处理什么投诉（右下）。**每个象限独立加载、独立缓存，弱依赖不互相拖累。**

*repository/db/model/order.go:12 BossID + Type; product.go:23 BossID + Num*

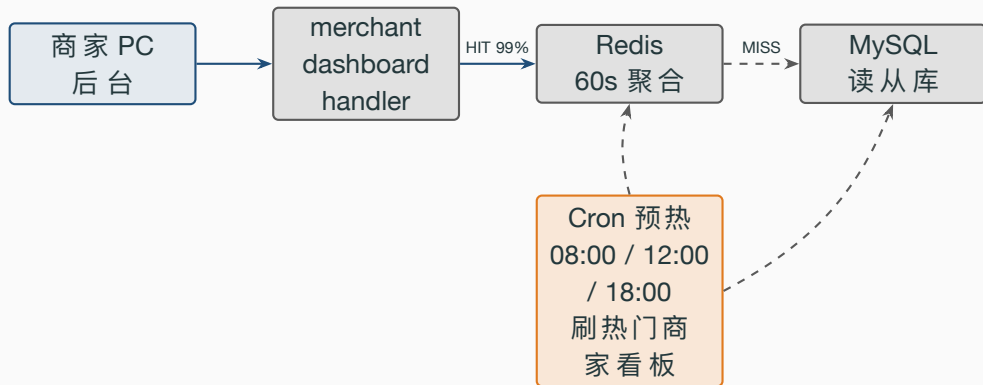
## 4 象限的数据源与计算口径

| 象限   | SQL/Redis 来源                             | 计算口径                  | 缓存策略      | 谁负责               |
|------|------------------------------------------|-----------------------|-----------|-------------------|
| 今日营收 | order(BossID, Money, Type=2, created_at) | SUM(Money) GROUP BY 日 | Redis 60s | service/merchant  |
| 待发货  | order(BossID, status=wait_ship)          | COUNT + 列出超 24h       | 列表 30s    | service/merchant  |
| 库存告警 | ZSET inv_alert:{boss}                    | ZRANGEBYSCORE 0 阈值    | 实时        | service/inventory |
| 客诉工单 | ticket(BossID, status='open')            | COUNT 未结              | 缓存 60s    | service/ticket (新 |

营收口径要讲清楚：只算已支付（**Type=2**）；退款的减回去（需要订单状态机 7 态完整落地后再加 Refunded 维度）。缓存粒度按商家：key 形如 merchant:dashboard:{boss\_id}:{date}，不同商家不会互相挤压。

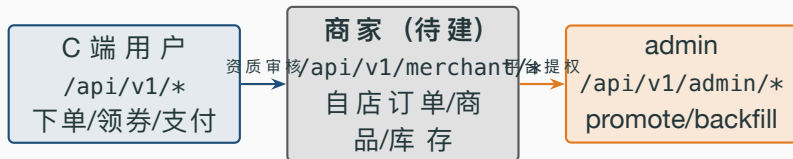
repository/db/model/order.go Type=2 已支付；订单状态机 7 态见 deck 09 v2

## 看板架构：读分离 + 三层缓存 + 异步预热



- **读分离**：所有查询打**从库**，避免占用下单链路的主库连接池。
- **60s 缓存**：商家可接受 1 分钟的数据延迟（不是炒股盘）；同时把 DB 压力砍到 1/60。
- **异步预热**：Cron 在三个高峰前把头部商家的看板提前算好，确保 p99 不抖。

## 今天只有两层墙，明天要加中间一层



| 角色       | 路径                 | 现状 | 备注             |
|----------|--------------------|----|----------------|
| user     | /api/v1/*          | ✓  | 默认放行           |
| merchant | /api/v1/merchant/* | 缺  | 中间一层待建         |
| admin    | /api/v1/admin/*    | 部分 | 已有 3 个 handler |

第三层 admin 已落地

`(RequireRole("admin"))`; 缺的是中间一层 **merchant**。 *routes/router.go:148-155 admin 子组;*  
*middleware/rbac.go:48-86 RequireRole*



## merchant 角色落地需要改的四处

1. repository/db/model/user.go:27--32: 新增常量 RoleMerchant = "merchant"。Role 字段已有, 无需 migration。
2. middleware/rbac.go: RequireRole 已是变长参数 (allowed ...string), 调用方传 "merchant", "admin" 即可。
3. routes/router.go: 新增 merchant := authed.Group("/merchant") + merchant.Use(middleware.RequireRole("merchant", "admin"))。
4. service/admin.go: 补一个 PromoteToMerchant, 参考 PromoteToAdmin:46--62 直接复刻; 记得调 InvalidateRoleCache。

四处改动各 5-15 行, 不需要任何 DB migration。整套路线图**两个 PR 就能合**。

*routes/router.go:148-155 现成模板; service/admin.go:46-62 promote 逻辑可复用*

## RequireRole + lookupRole: 30 秒缓存的取舍

Listing 2: *middleware/rbac.go:22-45 + :89-91*

```
22 var (
23     roleCache sync.Map
24     roleCacheTTL = 30 * time.Second
25 )
26 func lookupRole(ctx context.Context, userId uint) (string, error) {
27     if v, ok := roleCache.Load(userId); ok {
28         e := v.(roleCacheEntry)
29         if time.Now().Before(e.expires) { return e.role, nil }
30     }
31     u, err := dao.NewUserDao(ctx).GetUserById(userId)
32     if err != nil { return "", err }
33     role := u.Role
34     if role == "" { role = "user" }
35     roleCache.Store(userId, roleCacheEntry{role: role,
36         expires: time.Now().Add(roleCacheTTL)})
37     return role, nil
38 }
39 func InvalidateRoleCache(id uint) { roleCache.Delete(id) }
```

## Jaeger: 链路追踪解决的真实问题

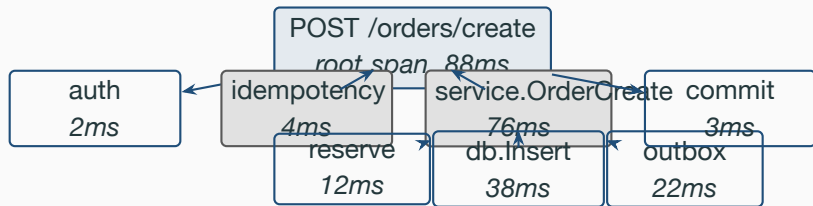
---

## 客服为什么需要 trace：从”客户骂街”到”5 分钟答完”

- **业务场景**：用户在客服群骂”我刚才那一单 5 秒没响应，钱扣了订单没出”。客服当场需要回答两个问题：(a) 钱有没有真扣；(b) 慢在哪一步、要不要赔偿。
- **没有 trace 怎么办**：客服只能”我帮您反馈技术” → 工单转 SRE → SRE 翻日志找用户 ID 找时间段 → 平均 2-6 小时。期间用户已经在朋友圈发截图。
- **有 trace 怎么办**：客户 App 报错弹窗带 `traceld` (产品需求) → 客户截图给客服 → 客服 Jaeger 输入 `traceld` → 看到 `POST /orders/create root 88ms` 含 4 个子 `span` → 5min 内答客户”已下单，物流单号晚 1 分钟生成”。
- **业务价值**：客服满意度的最强变量是**响应时间**（行业基准：5min 内首响 → 满意率 > 90%）。trace 把”客服找 SRE”链路砍掉。
- **所以下一帧的 span 树**：不是炫技图，是客服界面里的真实输入输出。

*middleware/track.go:11-29 traceld 透传；客户端弹窗带 traceld 是产品侧约定*

## 一次下单的 trace span 树



一棵树就把”慢在哪

一段”答清楚。**root 88ms** 中 **db.Insert** 占 **38ms**，是该接口下一步优化的着力点。

`service/order.go OrderCreate` 主路径；`track.WithSpan` 在子函数里开 `child span`

Listing 3: *middleware/track.go:11-29*

```
11 func Jaeger() gin.HandlerFunc {
12     return func(c *gin.Context) {
13         traceId := c.GetHeader("uber-trace-id")
14         var span opentracing.Span
15         if traceId != "" {
16             var err error
17             span, err = track.GetParentSpan(
18                 c.FullPath(), traceId, c.Request.Header)
19             if err != nil { return }
20         } else {
21             span = track.StartSpan(
22                 opentracing.GlobalTracer(), c.FullPath())
23         }
24         defer span.Finish()
25         c.Set(consts.SpanCTX, opentracing.ContextWithSpan(c, span))
26         c.Next()
27     }
28 }
```

- **第 3 行**：读 `uber-trace-id` header；格式 `<traceId>:<spanId>:<parentId>:<flags>`。
- **第 5–9 行**：有 `traceId` → 当前请求是别人调过来的，开 **child span**，root 留在上游。
- **第 10–13 行**：没 `traceId` → 当前请求是入口，自己开 **root span**。
- **第 14 行**：`defer span.Finish()` 必须有，否则 Jaeger 端会看到“未结束”灰色 trace。
- **第 15 行**：把 span 写回 `c.Context`，下游通过 `ctl.GetSpanContext` 开 **child span** ——这一步就是“链路”的串接。

*pkg/utils/track/track.go:61–78 GetParentSpan Extract*

## Jaeger 的开销：实测低于 5%

| 场景                                      | RPS    | p95    |
|-----------------------------------------|--------|--------|
| /ping 裸基线（含 ratelimit/cors/Jaeger 全中间件） | 64,254 | 3.51ms |
| /product/show 真业务（MySQL PK + 全中间件）      | 62,226 | 3.01ms |

- /ping 已经过了 Jaeger() (router.go:23 注册)，不是“无 trace 裸 RPS”。
- 商品详情 RPS 仅比 ping 低 **3.2%**，证明全链路 trace 在 const sampler 全量采样下，单机损耗 < 5%。
- 真把损耗摠住，还能从 const 切到 probabilistic 抽样（默认 0.001%），CPU 占用再降一个数量级。

*stressTest/REPORT.md* 链路压测表第 2、3 行；*pkg/utils/track/track.go:15-27 GetDefaultConfig const sampler*



## Jaeger 与 Skywalking 同时存在的取舍

| 维度        | Jaeger (OpenTracing)  | Skywalking    |
|-----------|-----------------------|---------------|
| 插桩方式      | 业务代码显式开 span          | agent 自动织入    |
| 学习曲线      | 中 (要懂 carrier/inject) | 低 (接入即用)      |
| 定制空间      | 高 (可任意标 tag)          | 中 (agent 决定)  |
| UI / 拓扑   | 单服务 trace 视角          | 全链路服务拓扑图      |
| gomall 用法 | 已落地, 主线               | 备选, docker 起着 |

**业务选型逻辑：**单服务调试用 Jaeger 够细；多服务上线后看拓扑用 Skywalking。两者不冲突，可以并存——当前 gomall 把两套基础设施都备齐，主推 Jaeger，留 Skywalking 做 SRE 大盘。 `cmd/main.go:44` 注释掉的 `//track.InitJaeger()` 留了切换开关

# Prometheus 业务 metrics + Grafana 大盘

---

## SRE 为什么需要 metrics：运营要“实时 GMV 大盘”

- **业务场景**：双 11 大促 0 点开闸，运营总监站在大屏前盯 GMV 增长曲线；SRE 同时盯下单 p99 和错误率。两边看的是**同一组指标的不同切面**。
- **运营关心**：当前 RPS、累计 GMV、商家活跃数、转化漏斗（曝光 → 加购 → 下单 → 支付）。→ `order_created_total + paydown_total` 加业务标签即可呈现。
- **SRE 关心**：下单 p99、错误率、限流命中率、熔断器状态。→ `order_create_duration_ms Histogram + ratelimit_hit_total + circuit_state`。
- **商家域负责人关心**：商家健康度——平均发货时长、退货率、客诉率。→ 业务指标，从 DB 聚合到 Grafana。
- **业务化设计原则**：指标命名带业务前缀（`gomall_order_*` / `gomall_payment_*`），标签是**业务语言**（`status=success/limited/cb_open`），不是**技术语言**（`http_code=429/503`）。这样运营也能读懂大盘。

## 要给业务接的 4 类 metrics

| metric                   | 类型        | 标签                     | 用途       | 来源代码                              |
|--------------------------|-----------|------------------------|----------|-----------------------------------|
| order_created_total      | Counter   | status, boss_id_bucket | 单数 / 转化  | service/order.go OrderCreate      |
| order_create_duration_ms | Histogram | status                 | p50/p99  | middleware/timing.go (新建)         |
| paydown_total            | Counter   | channel, status        | 渠道占比     | service/order_payment.go          |
| ratelimit_hit_total      | Counter   | scope                  | 限流命中率    | middleware/ratelimit.go SlidingW  |
| circuit_state            | Gauge     | route                  | 熔断器实时态   | middleware/circuitbreaker.go stat |
| idempotency_hit_total    | Counter   | route                  | 60002 命中 | middleware/idempotency.go         |

标签设计原则：**枚举值数量有限**。boss\_id 不能直接当标签（百万级 cardinality 会把

Prometheus 内存打爆），改成 boss\_id\_bucket（按 GMV 等级分 5 档）。

middleware/ratelimit.go SlidingWindow / middleware/circuitbreaker.go state；指标可在两个中间件里挂

Listing 4: *pkg/utils/metrics/metrics.go* 草案; 与 *middleware/timing.go* 配合

```
1 package metrics
2
3 import (
4     "github.com/prometheus/client_golang/prometheus"
5     "github.com/prometheus/client_golang/prometheus/promauto"
6 )
7
8 var (
9     OrderCreatedTotal = promauto.NewCounterVec(
10         prometheus.CounterOpts{
11             Name: "gomall_order_created_total",
12             Help: "订单创建累计计数",
13         }, []string{"status", "boss_bucket"})
14
15     OrderDurMs = promauto.NewHistogramVec(
16         prometheus.HistogramOpts{
17             Name: "gomall_order_create_duration_ms",
18             Help: "下单耗时 (毫秒) ",
19             Buckets: []float64{5, 10, 25, 50, 100, 250, 500, 1000}
```

- **promauto 而不是手 Register**: 自动注册到默认 Registry, 避免漏注册导致 metric 丢失。
- **Counter vs Histogram vs Gauge**: 单调递增用 Counter; 分位用 Histogram (buckets 是预定义的耗时阈值, 决定 p99 精度); 当前态用 Gauge。
- **Histogram buckets 选择**: 从 5ms 到 1000ms 八档, 参考 /orders/create 实测 p95  $\approx$  2.3ms (见 deck 10 压测表), 5ms 起步刚好覆盖 p95 + p99。
- **标签穷举**: status 取 success / fail / limited / cb\_open / idempotent\_hit, 5 个值; boss\_bucket 按 GMV 分 S / A / B / C / D, 5 个值。两个标签  $5 \times 5 = 25$  个时间序列, 可控。
- **怎么暴露**: 在 router 加 `r.GET("/metrics", gin.WrapH(promhttp.Handler()))`, Prometheus 默认 15s 抓一次。

Prometheus 官方建议 Counter / Histogram / Gauge 三类用法; stressTest/REPORT.md 第 1-3 行实测 p95 数据

# Grafana 看板：5 个核心面板

| 面板        | 数据源                                    | 阈值（黄 / 红）     | 告警接收人        |
|-----------|----------------------------------------|---------------|--------------|
| 订单总量趋势    | sum(rate(order_created_total[5m]))     | — / —         | 不告警          |
| 下单 p99 延迟 | histogram_quantile(0.99, ...)          | 200ms / 500ms | SRE          |
| 下单错误率     | rate(...status="fail"... ) / rate(...) | 1% / 5%       | SRE + 业务     |
| 限流命中率     | rate(ratelimit_hit_total[1m]) / RPS    | 0.5% / 2%     | SRE          |
| 熔断器开启次数   | changes(circuit_state==1[5m])          | 1 次 / 3 次     | SRE + 商家域负责人 |

阈值不是拍

**脑袋：**基线数据来自压测——ping 64K RPS / p95 3.5ms 是资源上限，下单 50K RPS / p95 2.3ms 是业务上限。生产把警戒线放在压测 p95 的 5–10 倍，留出余量。

*stressTest/REPORT.md* 链路压测表第 1、4 行 p95 数据；阈值倍率参考通用 SRE 实践

## ELK / EFK：日志集中化的工程化路线

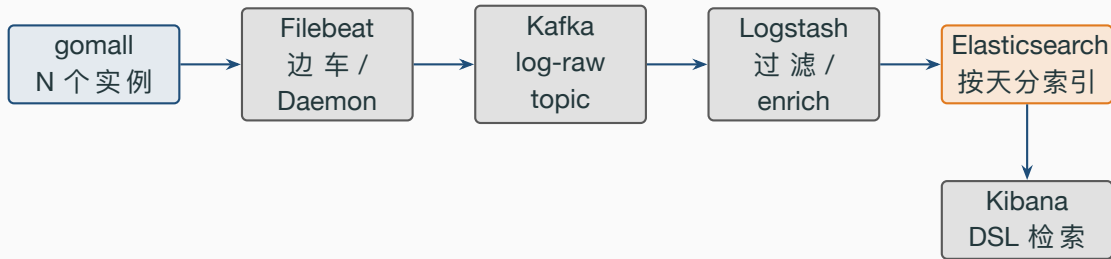
---



## 客服的“工具箱”：ELK 解决的三类客诉问题

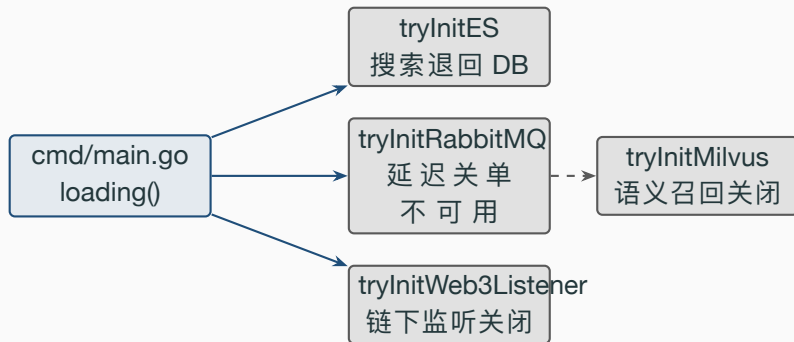
- **客诉类型 1：用户报“我那笔订单到底有没有出货”** → 客服在 Kibana 搜 `userId=10086 AND route="/orders/ship"` → 看到 5min 前发货 event 已写 `outbox` → 答客户“已发，物流单号 SF1234”。
- **客诉类型 2：用户报“我支付的钱去哪了”** → 搜 `userId=10086 AND route="/paydown"` 拉时间线 → 看到 `status=70002` 熔断 → 答客户“支付通道刚才异常，已退回，30 分钟内到账”，引用 README 客服话术表。
- **客诉类型 3：商家报“我新上的 SKU 搜不到”** → 搜 `level=warn AND msg="ES 不可用"` → 看到 ES 降级中 → 答商家“已切到 DB 路径，5 分钟内 ES 恢复就能看到”。
- **所以 ELK 不是“日志收集”，是客服的取证 + 沟通工具。** 日志 schema 必须可读：JSON + `traceld` + `userId`（脱敏）+ `route` + `status` + `msg` 五件套缺一不可。
- **对应 README 客服话术表：**业务码 70001/70002/50001/60002 + RP-EMPTY 都已经映射到话术，客服从 ELK 看到状态码 → 直接对照表回话。

## 规模化版本：Filebeat → Kafka → Logstash → ES



- 为什么中间加 **Kafka**：Filebeat 直连 ES，ES 抖动一次全站日志丢；Kafka 做 buffer，ES 可用性事故不会丢日志。
- **Logstash** 做什么：把 traceId 提取成顶层字段、userId 脱敏（保留前 3 + 后 3 位）、level=error 的同时复制到 alert 索引。
- 按天分索引：gomall-app-2026.05.16，便于按时间清理（保留 30 天，冷归档到 OSS）。

## gomall 启动期的四个 tryInit



- 四个外部依赖任意一个挂了，HTTP 主路径**照常起**：登录、浏览、下单、支付都不受影响。
- 用户最多失去某一项**增量能力**：搜不到语义结果、订单不自动延迟关闭，但不会“白屏 500”。

Listing 5: *cmd/main.go:64–73*

```
64 // tryInitES ES 不可用时静默跳过，搜索接口退回 DB 路径
65 func tryInitES(ctx context.Context) {
66     defer func() {
67         if r := recover(); r != nil {
68             util.LogrusObj.Warnf("ES 初始化失败，搜索退化: %v", r)
69         }
70     }()
71     es.InitEs()
72     initialize.InitSearch(ctx)
73 }
```

## tryInitES 逐行讲解：为什么不传 error

- **第 3–8 行 defer + recover**：底层 `es.InitEs()` 是老代码，连不上 ES 直接 panic。改 `InitEs` 签名风险大（连锁影响 search service），所以在外层包一个 **recover**。
- **为什么不返回 error 让 main 决定**：main 拿到 error 也只能写日志接着启——业务上“ES 挂”和“启动失败”是两件事，前者不应阻塞 HTTP 启动。
- **为什么用 Warnf 不是 Errorf**：error 级别会触发告警；这里是“已知降级路径”，不是异常。给 SRE 留 Warn 让他主动看，不要凌晨 3 点叫人起床。
- **相同套路用了 4 次**：`tryInitRabbitMQ`、`tryInitES`、`tryInitWeb3Listener`、`tryInitMilvus` ——同一种姿势，业务上是对“不可控外部依赖”的统一回应。

`cmd/main.go:54–97` 四个 `tryInit` 同构

## 业务码：现有 5 段 + 待加 80xxx 商家段

| 段     | 含义         | 现有 / 新增码            | 客服话术示例                |
|-------|------------|---------------------|-----------------------|
| 20xxx | 业务通用       | ERROR=20001         | ”请稍后再试”               |
| 30xxx | 鉴权         | ErrorAuth*          | ”登录已过期”               |
| 50xxx | 第三方        | ErrorOss=50001      | ”上传失败”                |
| 60xxx | 并发 / 幂等    | 60002 命中            | 静默（用户无感）              |
| 70xxx | 流量治理       | 70001 限流 / 70002 熔断 | ”正在排队 / 当前繁忙”         |
| 80001 | 店铺未审核      | 新增                  | ”您的店铺还在审核，1-3 工作日”    |
| 80002 | 操作非本店 SKU  | 新增                  | ”该商品不属于您的店铺”          |
| 80003 | 提现额度不足     | 新增                  | ”本月额度已满，下月 1 号刷新”     |
| 80004 | 发货超时改判即将触发 | 新增                  | ”您有 N 单超 72h 未发货”     |
| 80005 | 库存低于阈值     | 新增                  | ”SKU XXX 库存 < 阈值，请补货” |

编码原则

同 deck 10: **HTTP 200 + status 字段**。商家段隔离避免与 admin / C 端文案串号。

*pkg/e/code.go* 现有 5 段；*docs/slides/10-business-traffic-governance.tex* 话术映射模板

## 静默降级 + SLO + trace 驱动决策

---

## 静默降级的 5 条铁律（提炼自 cmd/main.go 现状）

1. **依赖必须是”增量能力”**：搜索语义召回、订单延迟关闭、链下事件、消息异步——这些都是”锦上添花”。下单 / 支付 / 鉴权**不能进降级名单**。
2. **兜底路径必须存在**：ES 挂 → InitSearch 不动，搜索退回 DB LIKE %keyword%；Milvus 挂 → semantic-search 接口短路返回 {items: []}。
3. **recover 必须配 Warn 日志 + traceId**：不能 silent。”sliently swallow + 0 日志”是事故制造机。
4. **recover 范围最小化**：只包 InitX，不包 r.Run。HTTP 主路径要 fail fast。
5. **对外可见性**：商家投诉”搜不到我新上的 SKU”时，SRE 一搜 Warn 日志 "ES 初始化失败" 就知道是哪种降级状态。

cmd/main.go:54-97 四个 tryInit 的同构姿势；repository/es/ 退化路径



## 静默降级的请求期版本：handler 内的弱依赖兜底

Listing 6: *service/product\_search.go* 草案；与 *cmd/main.go:64-73* 同构

```
1 // SearchProducts 弱依赖 ES，主路径返回 DB 结果，
2 // ES 命中则用 ES 的更优排序覆盖
3 func (s *ProductSrv) SearchProducts(
4     ctx context.Context, kw string) ([]*model.Product, error) {
5
6     // 1. 主路径：DB LIKE，必须返回
7     dbResult, err := s.dao.SearchByKeyword(ctx, kw)
8     if err != nil { return nil, err }
9
10    // 2. 锦上添花：ES 提供更好的排序
11    if !es.Available() {
12        log.WithCtx(ctx).Warnf("ES 不可用，搜索降级到 DB")
13        return dbResult, nil // 主路径已经够用
14    }
15    esResult, err := s.esRepo.SearchProducts(ctx, kw)
16    if err != nil {
17        log.WithCtx(ctx).Warnf("ES 查询出错，降级：%v", err)
18        return dbResult, nil
19    }
```

## 请求期降级套路逐行讲解

- **第 5–6 行**：DB 是底盘，先拿到结果。ES 用还是不用，是优化问题，不是功能问题。
- **第 9–10 行 `es.Available()`**：bool 缓存上一次健康检查结果（每 5s 更新），避免每次请求都打 ES。
- **第 11 行 `log.WithCtx(ctx).Warnf`**：traceld 自动注入，客服报“搜不到” → Kibana 搜 traceld 一眼看清是降级了。
- **第 14–17 行**：ES 报错时不 **return error**，因为 DB 已经给了答案，用户根本不需要知道 ES 有事。
- **第 18 行 `mergeResults`**：把 ES 的语义排序 + DB 的 keyword 全量融合，ES 命中段排前，未命中段按 DB 顺序排后——见 deck 21 v2 hybrid 排序。

*cmd/main.go:64–73 启动期同构；deck 21 v2 hybrid 排序公式*

## gomall 的 4 级 SLO 表与年宕预算（与 README 对齐）

| SLO 级别  | 例子接口                                 | 可用率    | 年宕预算      | p99 延迟  |
|---------|--------------------------------------|--------|-----------|---------|
| P0 核心交易 | /orders/create / /paydown            | 99.95% | 4h 22min  | < 500ms |
| P1 核心读  | /product/show / /orders/list         | 99.9%  | 8h 45min  | < 200ms |
| P2 营销秒杀 | /coupon/claim / /skill_product/skill | 99%    | 87h 36min | < 200ms |
| P3 信息类  | 轮播 / 分类 / 商家后台                       | 99%    | 87h 36min | < 1s    |

- 预算思想：SRE Book 经典——“可靠性不是越高越好，多出来的可靠性预算应当转成产品迭代速度”。预算是花的，不是省的。
- 为什么 P0 卡 99.95% 不卡 99.99%：99.99% 需要多 AZ + 蓝绿 + 主从切换演练，成本翻倍。99.95% 主从 + Redis + 静默降级即可达到。性价比对齐业务体量。
- 为什么商家后台 P3 = 信息类：商家可以等、用户不能等。把商家后台从 P0 降到 P3 是故意的资源调度：把 SRE 精力从“半夜修轮播”转移到“白天迭代下单链路”。
- 大促前 budget 预留：双 11 前一周冻结非紧急上线，把当月剩余 budget 留给大促当天的兜底操作（限流参数微调、熔断阈值微调、强制刷缓存等动作都会“花”预算）。
- budget 烧完怎么办：当月剩余 < 50% → 冻结非紧急上线；< 20% → 冻结所有上线；< 0% → 产品 team 全员复盘“为什么 SLO 守不住”。

README 业务承诺 SLO 表（P0 99.95% / P1 99.9% / P2 99% / P3 99%）；Google SRE Book Ch.4

## P0-P3 告警分级与响应 SLA

| 级别 | 触发条件示例                     | 响应 SLA | 通知方式       | on-call 行为   |
|----|----------------------------|--------|------------|--------------|
| P0 | 下单错误率 > 5%、支付完全不可用         | 5 分钟   | 电话 + IM 全员 | 立即上线，主管同时介入  |
| P1 | 下单 p99 > 1s 持续 10 分钟、ES 全挂 | 15 分钟  | 电话 + IM    | 主 on-call 上线 |
| P2 | 限流命中率 > 2%、熔断器 5 分钟 3 次    | 1 小时   | IM 通知      | 工作时间内排查      |
| P3 | Warn 日志聚集、商家后台慢            | 工作时间   | IM 静默      | 次日复盘         |

- **分级原则**：不是“看告警严重程度”，是“看用户感知 + 业务收入”。下单挂了影响 GMV，P0；商家后台慢 P3 给 BD 解释一下就行。
- **on-call 轮值**：每周一轮换，工作日 + 周末分两人；P0 双 on-call 自动同时呼叫。
- **post-mortem**：所有 P0 / P1 事故 7 天内必须出 blameless 复盘，沉淀到 wiki。

告警阈值与本 deck “Grafana 看板：5 个核心面板” 阈值表对齐

## P0-P3 的业务后果：每一级对应的 GMV 损失

| 级别 | 业务后果         | 估算数字（日 GMV 1000 万）         | 客户感知            |
|----|--------------|----------------------------|-----------------|
| P0 | 全站下单挂、GMV 归零 | 每 1 min 损失 7 千元            | App 弹”稍后再试”，跳竞品 |
| P1 | 核心读慢、转化率下降   | 每小时损失 5% 转化 $\approx$ 2 万元 | 列表转圈，加购流失       |
| P2 | 营销活动慢、ROI 损失 | 单场秒杀少卖 10%-20%，营销预算白烧      | 抢券失败客诉，运营道歉     |
| P3 | 信息类慢、体验小问题   | 几乎无 GMV 影响                 | 部分用户多等几秒        |

- 为什么把 P0 5 分钟卡死：日 GMV 1000 万 = 每分钟约 7 千元。5min 上线意味着单次事故损失上限约 3.5 万元（不含品牌口碑）。再松就是事故而非降级。
- 为什么 P3 进工作时间：商家后台慢、运营报表慢，客户感知弱、收入影响小。半夜 3 点叫 SRE 起来修轮播图，是 SRE 流失制造机（业内常说“on-call 倦怠”）。
- 告警分级 = 业务对故障的真实定价。不是 P 越多越严肃，是把人和钱花在影响最大的事故上。

GMV 估算口径与 README ”业务承诺 SLO” P0 99.95% / P1 99.9% / P2 99% / P3 99% 对齐

## SLO 燃尽预算 (Error Budget Burn Rate)

- **Burn rate** = 当前错误率 ÷ SLO 允许的错误率。
- 下单 SLO=99.99% → 允许错误率 0.01%。当前 5 分钟内实测错误率 0.05% →  
burn rate = 5。
- 阈值告警：
  - burn rate > 14.4: 1 小时烧完月预算 → P0
  - burn rate > 6: 6 小时烧完 → P1
  - burn rate > 3: 1 天烧完 → P2
  - burn rate > 1: 1 个月烧完 (即只是触线) → P3
- 为什么不用静态阈值: 静态阈值在高峰期容易误报、在低谷期容易漏报; burn rate 是相对预算的速率, 跨流量级别都准。

## 真实案例：从 trace 找到订单列表深分页问题

- **症状：**商家投诉“翻到第 100 页特别慢”。客服无 `traceld` 可贴，先用 Kibana 按 `route: "/orders/old/list"` 拉日志，发现 90% 请求耗时  $> 5s$ 。
- **用 trace 定位：**在 Jaeger 按 `op=GET /orders/old/list` 拉 p99 trace 一条，看到 **db.Query span 占 root 的 99%**，扫了 4M 行。
- **优化方案：**游标分页 + Redis 5min 首页缓存（见 PR #38）。
- **产出效果：**压测对照——旧版 **8.3 RPS / p95 15.95s**，优化版 **58,406 RPS / p95 5ms**。吞吐 7000x，延迟 3200x。
- **决策闭环：**trace 看出瓶颈 → PR 优化 → 压测对比 → 灰度 → Grafana 监控 p99 持续观察 1 周。

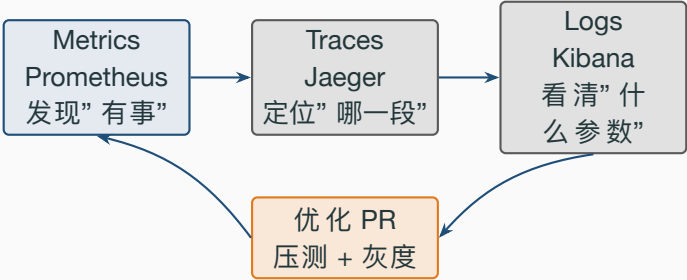
*stressTest/REPORT.md* 第 8、3 行；PR #38 游标分页

## 业务边界：商家后台 + 可观测不做什么

- 不做商家自助看板 UI：BossID API 提供数据，前端由商家自己接 Web/App 或对接平台 BI；招商 pitch 场景用 Postman / Swagger。前端 UI = 独立路线图。
- 不做自定义告警规则：商家不能自己写“我的销量低于 X 就告警我”。规则由 SRE / 商家域负责人统一维护，避免商家滥用告警把 SRE 拖死。
- 不做日志归档冷存：当前只保 30 天热索引；7 天以上 OSS 归档 + Glacier 长存是路线图。法务合规口径还没拍板。
- 不做 SaaS 化多商户隔离：当前 BossID 是逻辑隔离（SQL 加 AND boss\_id=?），不是物理隔离（独立 schema / 独立 RDS）。SaaS 化路线图要再独立讲。
- 不做平台计费 / 抽佣：商家结算 settlement 表只是聚合，没有真发钱。”按 GMV 抽 5%”的财务系统对接由 wallet 服务做（路线图）。
- 不做招商审核流程：阶段 2-4（OCR / 人工审核 / 试运行）只有文档草案，没有真表 merchant\_apply 和真服务。
- 不做客服工单系统：客服当前用 ELK + 业务码表 + 话术对照 + 手工 IM 沟通。ticket 表 + 工单 SOP 是路线图。



# 五角色共用一套底层信号 + 三件套闭环



| 角色    | 关心的事                     | 看哪一层                      |
|-------|--------------------------|---------------------------|
| C 端用户 | 我下单成功了吗、钱到哪了             | App 错误码 + traceId (产品提示)  |
| 商家    | 卖了多少 / 待发几单 / 何时到账       | 商家后台 + settlement (路线图)   |
| 运营    | 实时 GMV / 商家健康度           | Prometheus + Grafana 业务大盘 |
| 客服    | 单个 traceId / 用户做过什么      | Kibana + Jaeger 联合        |
| SRE   | p99 / 错误率 / SLO 预算 / 熔断态 | Grafana + Jaeger + 烧穿告警   |

号 (JSON 日志 + trace + Prom 指标), 差别只在聚合维度和查询入口。三件套缺一不可: metrics 知道有事不知道在哪、trace 看个体不看趋势、log 看细节不看总体。发现 → 定位

## 面试 Q&A (1/2): 架构与权限

**Q1:** BossID 都在了, 为什么不直接给商家开 admin?

A: admin 看全站数据, 商家只能看自店; 权限边界不能用”先开放再监控”做。  
merchant 角色 + RequireRole("merchant") 才是正确解。[middleware/rbac.go:48-86]

**Q2:** merchant.Group 为什么不在路由层校验 boss\_id?

A: boss\_id 从 ctl.GetUserInfo(ctx).Id 取, 绝不进 body/query。handler 入参故意不暴露该字段——”入参里没有 = 没法越权”是设计层面的防御。[pkg/utils/ctl/ctl.go]

**Q3:** roleCache 为什么 30s 不是 5min?

A: 商家被提权后等 5min 才生效, 谈判桌上没法解释。30s 是”权限漂移可接受 + 回源 DB 砍 30 倍”的平衡点; 提权时 InvalidateRoleCache 让当事人立即生效。  
[middleware/rbac.go:22-45]

**Q4:** 商家入驻为什么要先 OCR 再人工审核?

A: OCR 即时返回, 过滤掉资质号不合规、姓名不一致的 80% 提交, 让人工只看真案子。审核 SLA 48h, 人工资源精确投放。[service/ocr.go 待建]

**Q5:** 商家看板 4 角限为什么独立加载?

## 面试 Q&A (2/2): 可观测性与降级

**Q7:** Jaeger 和 Skywalking 同时存在, 是不是重复建设?

A: 单服务调试 Jaeger 细, 多服务拓扑 Skywalking 直观。两者数据模型相同 (trace+span), 并存不冲突。gomall 主推 Jaeger, 留 Skywalking 做 SRE 大盘。  
*[cmd/main.go:7]*

**Q8:** Prometheus 的 boss\_id 标签为什么不能直接用?

A: Prometheus 标签 cardinality 必须有限。百万级 boss\_id 会把内存撑爆。改成 boss\_bucket (按 GMV 分 5 档), 时间序列数 = 标签笛卡尔积, 可控。  
*[pkg/utils/metrics/metrics.go 草案]*

**Q9:** Filebeat 直连 ES 不行吗, 为什么还要 Kafka?

A: Filebeat 直连 → ES 抖动一次全站日志丢。Kafka 做 buffer, ES 故障期间日志先进 Kafka, 恢复后由 Logstash 重放。可用性 ≫ 简洁性。*[docker-compose.yml:44-99]*

**Q10:** tryInitES 为什么用 recover 不传 error?

A: 底层 es.InitEs() 老代码 panic 风格, 改签名牵连大。外层 recover + Warnf 是“启动期不可控依赖”的统一回应, 与请求期熔断 (70002) 互补。*[cmd/main.go:64-73]*

# 代码位置一览 i

**BossID 散落字段** *model/order.go:12, product.go:23, cart.go:10, favorite.go:12*

**RBAC 中间件** *middleware/rbac.go:22–45, :48–86, :89–91*

**Jaeger middleware** *middleware/track.go:11–29*

**Trace utility** *pkg/utils/track/track.go:15–78*

**Skywalking agent** *cmd/main.go:7 import* 副作用

**logrus 初始化** *pkg/utils/log/logger.go:14–41*

**静默降级（启动期）** *cmd/main.go:54–97* 四个 tryInitX 同构

**静默降级（请求期）** *service/product\_search.go* 草案；*deck 21 v2 hybrid*

**merchant.Group 草案** *routes/router.go* *merchant := authed.Group("/merchant")*（本 deck p.10）

**merchant API 7 接口** *api/v1/merchant\_\*.go* 待建 7 个 handler

**商家入驻申请** *model merchant\_apply*（待建）；*service/merchant\_apply.go*

**Prometheus metrics** *pkg/utils/metrics/metrics.go* 草案；*middleware/timing.go*

**ELK / EFK 落地** *docker-compose.yml:44–99* ES+Kibana；Filebeat/Kafka/Logstash 路线图

**admin promote 模板** *service/admin.go:46–62 PromoteToAdmin*

**admin 路由组** *routes/router.go:157–164 admin 子组三接口*

**CircuitBreaker 配置** *routes/router.go:122–130 paydown*

**压测锚点** *stressTest/REPORT.md: ping 64,254 / product 62,226 / 订单优化 58,406 vs 旧 8.3 RPS*

配套：18-merchant-business.tex（商家旅程） / 10-business-traffic-governance.tex（流量治理） / 09 v2  
订单状态机 / 13 列表性能 / 21 v2 AI 检索